`ntpstats`: an open toolkit for validating, evaluating and studying network time synchronisation

Thiago de Freitas
Independent researcher
ORCID 0009-0006-7749-1401 GitHub @thiagodefreitas

Preprint, 11 October 2026
Software version 3.7.0, doi:10.5281/zenodo.23070521

Abstract

The quality of network time synchronisation is a measurement problem: how far a clock is from its reference, how stable it is, how much of the error the network contributes, and whether another algorithm would do better. In practice, each of these questions is answered with separate tools and ad hoc scripts, which makes results from different deployments and different papers hard to compare. We describe `ntpstats`, an open-source Python toolkit that joins the steps of such an evaluation. It reads the logs of current time daemons (ntpd, NTPsec, chrony, linuxptp, ntpd-rs), packet captures of NTP, PTP, NTP over PTP and client-server PTP, GNSS receiver and time-transfer data, laboratory counters and network-simulator output, and maps them onto one data model with a single sign convention. On this model it computes the frequency-stability statistics of NIST SP 1065 with confidence intervals from the exact equivalent degrees of freedom of the discrete power-law noise model, the time-error metrics of ITU-T G.8260, power-law noise fits, N-cornered-hat separation and holdover predictions. A research bench scores clock-synchronisation estimators against simulated clocks with known truth, including replayed network delays, PTP exchanges and chains of boundary clocks. We report the validation of the statistics against the published NIST test suites (largest relative deviation 2.8×10^{-7} at seven printed digits) and against Stable32's own output for eleven statistics, a Monte Carlo study of interval coverage, a benchmark of seventeen reference estimators, among them closed-loop models of ntpd's clock discipline and of the NIST time servers' algorithms, a study of time-error accumulation in boundary-clock chains, and a re-examination, on its original data, of the 2012 thesis in which the toolkit began. All results are reproducible from seeded scripts.

Keywords: network time synchronisation; NTP; PTP; frequency stability; Allan deviation; time error; clock-synchronisation algorithms; reproducible research.

1 Introduction

Packet networks distribute time to almost every computer, to mobile networks, to trading systems and to power grids. Most hosts use the Network Time Protocol (NTP) [1, 2], increasingly authenticated with Network Time Security (NTS) [3]; telecom and industrial networks use the Precision Time Protocol (PTP) of IEEE 1588 [4]. The protocols have matured, and the accuracy that is achievable now ranges from milliseconds over the Internet to nanoseconds with hardware time stamps and on-path support. Assessing that accuracy has not become easier. An operator who wants evidence that a fleet stays within a regulatory limit, an engineer qualifying a boundary clock, and a researcher proposing a new synchronisation algorithm all need the same chain of steps: extract the offsets and delays from what the system recorded, place them on a common time base and sign convention, compute statistics with defensible uncertainties, and compare the result with a limit or with an alternative.

These steps are well understood individually. The statistics of frequency stability go back to Allan [5] and Barnes et al. [6], and are collected in NIST SP 1065 [7] and IEEE Std 1139 [8]. Packet-network

time error is defined in ITU-T G.810 and G.8260 [9, 10], with limits in G.8271.1 and G.8273.2 [11, 12]. Synchronisation algorithms are documented in the NTP specification [2] and in the research literature [13–15]. What is missing is a common, open implementation that connects the data that systems actually produce with these definitions. Stability programs such as Stable32 [16], TimeLab [17] and the `allantools` library [18] expect a clean, uniformly sampled phase or frequency record; daemons and their plotting tools [19] report their own state rather than evaluate it; PTP time error is usually measured with vendor instruments and closed analysis software. Papers on synchronisation algorithms tend to come with one-off evaluation code, so their numbers rarely share definitions or test conditions.

`ntpstats` addresses this gap. It began in 2012 as a Google Summer of Code project for the NTP Project of the Network Time Foundation and as the software of the author's undergraduate thesis [20], and was rewritten from 2026 as a validation, evaluation and study toolkit. This paper describes the design of version 3.7.0 and reports its validation and a set of experiments that the toolkit makes reproducible. The contributions are:

(i) a data model and input layer that read 26 built-in formats, from daemon logs and packet captures (NTP, PTP, NTP over PTP [21], client-server PTP) to GNSS receivers, laboratory counters, BIPM and IGS products, Internet measurement platforms and network simulators, with one documented sign convention (Section 3);

(ii) implementations of the NIST SP 1065 stability statistics with confidence intervals from the exact equivalent degrees of freedom (EDF) of the discrete power-law noise model, together with ITU-T time-error metrics, power-law noise fitting, N-cornered-hat separation and holdover prediction (Section 4);

(iii) a research bench that scores synchronisation estimators against simulated clocks with known truth, including replayed network delays, exchange-level PTP and boundary-clock chains, and seventeen reference estimators (Section 5);

(iv) a validation against published reference values, a Monte Carlo study of interval coverage and a comparison with the output files of Stable32 (Sections 7 and 9), and a protocol and command for checking a disciplined clock against an independent reference;

(v) experiments on estimator accuracy, PTP clients, boundary-clock chains and public servers, and a re-examination of the 2012 thesis against known truth, all generated by seeded scripts published with the paper (Section 8).

2 Background

2.1 Two-way time transfer and its limit

An NTP client sends a request at its time T_1; the server receives it at T_2 and answers at T_3; the client receives the answer at T_4. The offset and the round-trip delay are

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}, \qquad \delta = (T_4 - T_1) - (T_3 - T_2), \tag{1}$$

with θ the reference minus the local clock [2]. If the one-way delays are d_f and d_b, the measured offset differs from the true one by $(d_f - d_b)/2$. Because $d_f, d_b \geq 0$ and $d_f + d_b = \delta$, the error of a single exchange is bounded by $\delta/2$, and the true offset lies in $[\theta - \delta/2, \theta + \delta/2]$. The bound is reached when all of the delay is on one side. A constant asymmetry cannot be observed with two-way time stamps at all: it biases every estimate by half its value. Queueing, on the other hand, varies from packet to packet, and the exchanges with the smallest delay are the least affected. This is the idea behind the NTP clock filter [2], the regression of chrony [22], feed-forward clocks [13] and the convex-hull approach of Huygens [14]. PTP uses the same equations, with one-way Sync messages and separate Delay_Req exchanges in the end-to-end delay mechanism, and adds a correction field in which transparent clocks report the queueing

they measured [4]. RFC 10030 carries NTP messages in PTP event messages so that NTP can benefit from PTP hardware time stamping and from these corrections [21].

2.2 Frequency stability and its uncertainty

Clock noise is commonly described by the power-law model of the one-sided spectral density of fractional frequency, $S_y(f) = \sum_{\alpha=-2}^{2} h_\alpha f^\alpha$, whose five terms are white and flicker phase modulation (PM, $\alpha = 2, 1$), white and flicker frequency modulation (FM, $\alpha = 0, -1$) and random-walk FM ($\alpha = -2$) [6]. The Allan deviation and its overlapping, modified, Hadamard, total and Theo variants [5, 23–26] estimate the stability at averaging time τ from phase data x sampled at interval τ_0; NIST SP 1065 gives their definitions and test data [7]. Each estimate is a random variable. Its confidence interval follows from a χ^2 distribution whose equivalent degrees of freedom depend on the estimator, the averaging factor $m = \tau/\tau_0$, the record length and the noise type [27]. The noise type can be identified from the data with the lag-1 autocorrelation [28]. A widely used shortcut, an error bar of $\sigma_y/\sqrt{n}$ for n terms, ignores the correlation between overlapping terms; Section 7.2 quantifies how much that matters.

2.3 Time error in packet networks

In telecom practice, the time error is $\mathrm{TE}(t)$ = local − reference. ITU-T G.8260 defines the maximum absolute time error max|TE|, the constant time error cTE, and the dynamic time error dTE, split by a low-pass filter (0.1 Hz) into a low-frequency part $\mathrm{dTE_L}$, characterised by the maximum time interval error (MTIE) and the time deviation (TDEV), and a high-frequency part $\mathrm{dTE_H}$ [10]. G.8273.2 sets per-clock limits for boundary clocks in classes A to D, and G.8271.1 sets network limits such as 1.1 µs max|TE| at the end of a chain for applications that need 1.5 µs end-to-end [11, 12].

2.4 Related tools

Stable32 [16] and TimeLab [17] are the reference desktop programs of the time and frequency community; `allantools` [18] provides the classical deviations in Python. They analyse clean phase or frequency records and do not read daemon logs or packet captures, nor address network effects or the time-error metrics of packet networks. NTPsec's `ntpviz` [19] plots percentiles of ntpd statistics files. OMNeT++ with INET [29, 30] and ns-3 [31] simulate networks, including PTP models, but leave the statistical evaluation to the user. Algorithm papers such as Huygens [14], Sundial [15] and RADclock [13] report their own evaluations under their own conditions. `ntpstats` is designed to interoperate with these tools rather than replace them: it reads and writes Stable32 data files, offers an `allantools`-compatible interface, and reads and writes OMNeT++ vector files and ns-3 text and captures.

3 Data model and inputs

3.1 One series type and one sign convention

Every input becomes a `TimeSeries`: POSIX time stamps t_i, offsets θ_i in seconds, optional per-sample columns (round-trip delay, dispersion, frequency, servo state, PTP correction, GNSS quantisation error, and others) and a metadata dictionary that records the source, the peer and any assumption made while parsing. Offsets always mean *reference minus local*, the convention of ntpd [2]. Implementations disagree on this point: chrony's tracking and statistics logs and linuxptp report local minus reference, RIPE Atlas reports local minus server, and the time-error metrics of ITU-T use local minus reference. The parsers convert on import, following each implementation's documentation, and the conversion is tested on the line layouts given in that documentation. A consistency test simulates one NTP peer and writes it as ntpd `peerstats`, ntpd `rawstats`, chrony `measurements.log` and a packet capture; all four must read back identically.

Table 1: Input formats of `ntpstats` 3.7.0, grouped by origin. All are detected from content except Stable32 files and bare TICC interval readings.

Origin	Formats
NTP daemons	ntpd/NTPsec `loopstats`, `peerstats`, `rawstats`; chrony `tracking`, `measurements`, `statistics`, `refclocks`; Windows `w32tm`
PTP	linuxptp (`ptp4l`, `phc2sys`, `ts2phc`)
Captures	pcap and pcapng: NTPv3 to v5, NTP over PTP, interleaved NTP, PTPv2/v2.1 (UDP, Ethernet, one- and two-step, E2E and P2P), CSPTP
GNSS and laboratory	u-blox UBX, gpsd JSON, TAPR TICC, CGGTTS, RINEX clock, BIPM Circular T, Stable32 phase and frequency files
Monitoring	Prometheus range queries, clock-error bound windows (ClockBound [42] and CSV), the ntpstats open interop dataset
Internet measurements	RIPE Atlas NTP results, NTP Pool monitor logs
Generic	CSV, Parquet, declarative instrument profiles (TOML), the 2012 prototype's log

Irregular logs are placed on a uniform grid at their median sampling interval τ_0. Grid points inside gaps longer than a configurable multiple of τ_0 are left empty and every estimator term that touches them is dropped, so gaps are never bridged by interpolation, which would add exactly the kind of smooth noise that the statistics are meant to measure. Time stamps from captures and bound windows are parsed as exact integers or decimals, because a double-precision number cannot hold nanoseconds at current POSIX epochs.

3.2 Inputs

Table 1 groups the 26 built-in formats. Formats are detected from content, so most commands need no format flag. Daemon logs cover the three NTP implementations in common use and the linuxptp stack [22, 32, 33]. Live sampling is available for chrony, ntpd, ntpd-rs [34], ptp4l, Meta's `ptpcheck` [35] and gpsd [36].

The capture reader decodes NTP versions 3 to 5 and IEEE 1588 version 2 and 2.1 over UDP and Ethernet. For PTP it reconstructs the offset and mean path delay of each master and slave pair from Sync, Follow_Up, Delay_Req and Delay_Resp (or the peer-delay messages), applying the correction fields, and it takes the UTC offset from Announce messages. For NTP over PTP [21] it applies the corrections of the request and the response as the RFC specifies,

$$\theta_c = \theta + \frac{c_{\mathrm{rs}} - c_{\mathrm{rq}}}{2}, \qquad \delta_c = \delta - (c_{\mathrm{rs}} + c_{\mathrm{rq}} - d_{\mathrm{rs}} - d_{\mathrm{rq}})(1 - 10^{-4}), \tag{2}$$

where c are the network corrections and d the frame durations, and it rejects corrections that the RFC says must not be applied. Client-server PTP exchanges (CSPTP), as implemented experimentally in ntpd-rs, are reconstructed with all four time stamps and their correction fields. Interleaved NTP responses [37] are recognised, and the offset is recomputed with the precise transmit time stamp carried in the following response.

GNSS inputs include u-blox UBX messages [38] (receiver clock bias and the quantisation error of the time pulse), gpsd's JSON reports of each PPS edge as both GNSS and system time, the TAPR TICC time-stamping counter [39], CGGTTS common-view files [40], RINEX clock products and BIPM Circular T. Measurements from RIPE Atlas [41] and the NTP Pool monitors can be read as they are downloaded. OMNeT++ vector files are read and written, and an INET clock becomes its time error against simulation time.

4 Statistics, metrology and time error

4.1 Stability estimators

`ntpstats` implements the Allan deviation in its standard and overlapping forms (ADEV, OADEV), the modified Allan deviation (MDEV), the time deviation (TDEV), the overlapping Hadamard deviation (HDEV), the total, modified total, time total and Hadamard total deviations (TOTDEV, MTOT, TTOT, HTOT), Theo1, TheoBR and TheoH, and the maximum and RMS time interval error (MTIE, $\mathrm{TIE}_{\mathrm{rms}}$) [7, 9, 24–26]. Each estimator has a literal implementation of its published sum in the test suite against which the vectorised implementation is checked. The total deviations are bias-corrected for the noise type identified at each averaging time, as NIST SP 1065 prescribes: TOTDEV by $1 - a\,\tau/T$, with $a = 1/(3\ln 2)$ for flicker FM, $3/4$ for random-walk FM and 0 otherwise [7, sec. 5.11], MTOT and TTOT by the MTOT bias factor, and HTOT by its own factor. The handbook's tables contain the corrected values; the raw values are available on request. Stable32 does not apply all of these corrections in its output files (Section 9). MTOT and the Theo family cost $O(Nm)$ operations per averaging factor; above a configurable work limit they average every k-th subsequence, with $k \le m$, and the result records which stride was used, so that a million-sample record remains practical without silently changing the definition.

4.2 Confidence intervals and noise identification

At each τ the dominant noise type is identified with the lag-1 autocorrelation method [28]. The EDF of ADEV, OADEV, MDEV, TDEV and HDEV is then computed exactly for the discrete power-law model: the estimator is written as a linear filter on the phase, the filter is convolved with the fractional-integration filter that generates the noise [43, 44], and the EDF follows exactly from the autocovariance of the combined filter output. This is the discrete-time counterpart of the method of Greenhall and Riley [27], whose continuous-time model assumes that the phase is averaged over each sample interval, whereas clock offsets are instantaneous samples. Intervals are two-sided χ^2 intervals at the chosen confidence level. Stability results can be checked against mask files with the upper confidence bound, for acceptance tests.

4.3 Noise model, spectra, N-cornered hat and holdover

The five coefficients h_α are fitted by weighted non-negative least squares to the variances, with weights from the EDF, and with every subset of noise types tried; the subset with the lowest Bayesian information criterion [45] is kept, so that a noise type is reported only when the data need it. A linear frequency drift can be fitted at the same time. Uncertainties come from a parametric bootstrap that re-simulates and refits the model. Power spectral densities of phase or frequency are computed per gap-free segment with Welch's method [46] or sine multitapers [47, 48], and can be expressed as single-sideband phase noise $\mathcal{L}(f)$ [8].

With three or more clocks and no better reference, only differences can be observed. The toolkit separates individual stabilities with the three-cornered hat [49], its least-squares generalisation to N sources, and the Groslambert covariance, which for three sources gives the same estimate with a different uncertainty model [50]. Negative variance estimates, which occur when sources share noise, are reported with an upper limit rather than hidden.

Holdover prediction answers how long a clock stays within a limit if its reference disappears now. The clock is assumed to continue on the frequency (and optionally the drift) fitted over a window before the loss; the variance of the resulting time interval error is computed exactly for the fitted power-law model, including the error of the fitted frequency and drift, and the bootstrap models of the noise fit are mixed so that the uncertainty of the model itself enters the envelope. The same model answers how often a client must poll: between polls a client runs on the phase and frequency it has estimated, so its error just before the next poll is that of a holdover of one poll interval. For each candidate interval the log is decimated to it and the holdover error is predicted, so that the fewer measurements of a longer interval count against it; the result is the longest interval that meets a target.

4.4 Network and time-error metrics

For two-way measurements the toolkit reports the delay floor and percentiles, the share of packets near the floor, the offset spread of floor packets, an asymmetry indicator, and the floor packet percentage of G.8260 [10]. For time error it computes max|TE|, cTE (also as the worst mean over windows), the filtered $\mathrm{TE_L}$ through a first-order 0.1 Hz low-pass, and $\mathrm{dTE_L}$ and $\mathrm{dTE_H}$ with their peak-to-peak values and the MTIE and TDEV of $\mathrm{dTE_L}$. The filter restarts after gaps and its settling time is excluded. When the sampling interval is too long for the filter to separate $\mathrm{dTE_L}$ from $\mathrm{dTE_H}$, $\mathrm{dTE_H}$ is reported as unavailable, and a limit on it is reported as not checked rather than passed. No standards text is bundled: limits and masks are supplied by the user, and the exit status of the command reports a violation so that it can gate a continuous-integration pipeline.

4.5 Assurance

For regulated users (for example the clock-synchronisation requirements of MiFID II [51]) the audit command computes a per-sample upper bound on the distance to the reference,

$$b_i = |\theta_i| + \frac{\delta_i}{2} + \left(\frac{D_i}{2} + E_i\right) + u_{\mathrm{ref}}, \tag{3}$$

from the measured offset, the worst-case path asymmetry, the upstream root delay D_i and root dispersion E_i reported by the server, and a declared uncertainty of the reference at the top of the chain. Time is cut into windows, each sample covers the time to the next one up to a maximum gap, and anything beyond that counts as unmonitored, never as compliant. Every assumption, for example a term that was not available and was set to zero, is stated in the report, and the report carries the SHA-256 hashes of its inputs. The same machinery validates clock-error bound services such as ClockBound [42]: each published window [earliest, latest] is classified as containing the reference, violating it, or indeterminate given the reference's own uncertainty. Change-point detectors locate phase steps, spikes, frequency changes (a standardised CUSUM [52] with binary segmentation) and changes of the delay floor, and mark each offset change by whether the path changed at the same time.

5 The research bench

5.1 Simulation with known truth

A scenario describes a local oscillator, one or more servers and the network paths to them. The oscillator has a frequency offset, a drift, white, flicker and random-walk FM and PM noise generated by fractional integration [43], and an optional sinusoidal temperature cycle. Each one-way path has a fixed propagation delay plus queueing that is exponentially distributed with a given mean and occurs with a given probability (the load), which gives the floor-plus-tail delay distribution of real networks. Events change a path over time: a route change steps the propagation delay, congestion scales the queueing, an outage drops packets. Servers can be biased (falsetickers) or step. Every simulated exchange is returned together with the true offset, so that an estimator can be scored on its error rather than on its own reported statistics.

Real networks enter through trace replay. From any two-way measurement, the per-direction delays are $\delta/2 \pm \theta$, which still contain the true offset between the two clocks; the toolkit removes it under one of three stated assumptions (the minimum-delay exchange of each window is symmetric, a constant frequency offset fitted by the Theil–Sen estimator [53, 54], or ends already synchronised). The resulting delay sequence is replayed in order or resampled with a moving-block bootstrap [55], which keeps the short-term correlation of both directions, under a simulated clock. The absolute asymmetry of a path cannot be recovered from two-way time stamps; the replay assumes equal floors unless a known asymmetry is given.

With `protocol = "ptp"`, a scenario produces the end-to-end delay mechanism of IEEE 1588 instead of NTP exchanges: Sync messages at one rate, Delay_Req exchanges at another, each with its own delay and time-stamp noise, and a fraction of the queueing corrected by transparent clocks.

Table 2: Reference estimators of the bench. "Multi" estimators combine several servers.

Name	Algorithm
`raw`	the measured offsets
`kalman`, `kalman-dw`	two-state (phase, frequency) Kalman filter [61]; `-dw` weights each sample by its queueing delay
`rts-dw`	Rauch–Tung–Striebel smoother [62] of the delay-weighted model (offline)
`mindelay`	NTP clock filter: minimum delay of the last eight samples [2]
`regression`	chrony-style weighted regression with the window sized by a runs test [22]
`feedforward`	RADclock-style: rate from low-delay packets over a long baseline [13]
`hull`	Huygens-style [14]: the line with the largest margin between the bounds $\theta \pm \delta/2$ of a causal window of 64 exchanges
`ptp4l`	linuxptp-style slave [32]: moving-median path delay, PI or linear-regression servo
`sptp`	SPTP-style client [56]: complete exchanges, delay-outlier rejection, PI servo
`rfc5905` (multi)	clock filter, then selection, clustering and combining of RFC 5905 [2, 63]
`median` (multi)	median of the clock-filter outputs of all servers
`kalman-combine` (multi)	per-source Kalman filters, interval intersection, inverse-variance mean
`ntpd`	ntpd 4.2.8's clock discipline in closed loop [57, 58]: clock filter with popcorn suppressor, state machine, hybrid PLL/FLL, slew bound
`ntpd-rfc`	the same with the constants and gain formulas of the RFC 5905 appendix [2]
`lockclock`	Levine's frequency-lock loop of the NIST time servers [59]
`levine-kalman`	the same with Levine's scalar Kalman time estimate [60]

5.2 Reference estimators

Seventeen estimators are registered (Table 2). They are written from the published algorithms, not from the daemons' code: `ptp4l`, for example, models the linuxptp slave with its default moving-median delay filter of ten samples and the PI servo with linuxptp's default gains for hardware time stamping, and `sptp` models the approach of Meta's Simple PTP client [56]. The clock-discipline models steer a clock in closed loop the same way. `ntpd` follows the reference implementation [57, 58]: the clock filter, including its popcorn spike suppressor and the unsorted register it uses during the start-up clamp, the state machine, the hybrid PLL/FLL and the 500 ppm bound on the total slew, with the time constant fixed at the measurement interval, as with equal minimum and maximum poll; `ntpd-rfc` replaces its constants with those printed in the RFC 5905 appendix. `lockclock` follows section 6 of [59]: a five-point majority vote per cycle, a time-adjustment mode while the residual exceeds 3σ, then a frequency estimate $y = \delta x/T$ averaged as $\bar{y} = (y + k\bar{y})/(k + 1)$ over cycles of about 1000 s with $k = 1$, the network configuration of the paper. `levine-kalman` corrects only the fraction $\sigma_o^2/(\sigma_o^2 + \sigma_t^2)$ of each residual [60], with σ_o^2 estimated online because the bench has no separate reference to calibrate it against. Third-party estimators are added by subclassing one class or through a Python entry point.

5.3 Scoring

Each estimate is compared with the true offset after a warm-up period (30 minutes by default), and the bench reports the RMS error, the bias, the 95th percentile and the maximum of the absolute error, the MTIE over one hour and the run time, over several seeds. Single-server estimators are scored on the first server of a scenario, multi-server estimators on all of them. A scenario file and a list of seeds fully determine a benchmark.

5.4 Boundary-clock chains

A separate simulator models a PTP grandmaster followed by N boundary clocks. Each boundary clock has its own oscillator (a frequency offset drawn within ±2 ppm, white and random-walk FM), runs the end-to-end delay mechanism with linuxptp's moving-median delay filter, and disciplines its clock with either the PI servo or a linear-regression servo modelled on linuxptp's. Links have a propagation delay,

an optional asymmetry, time-stamp noise (4 ns RMS per time stamp by default) and optional packet delay variation. The output is the time error of every node and the time error added by every hop, with the G.8260 metrics, checked against per-hop limits and an end-to-end budget.

5.5 Protocol clients

To measure servers rather than logs, `ntpstats` includes clients for SNTP (NTPv4) with a random transmit time stamp, NTS [3] including the server and port negotiation records and cookie renewal, the NTS pool extension that assigns distinct servers to one client [64], NTPv5 as in draft 09 [65] including the upgrade probe from NTPv4, the interleaved mode of RFC 9769 [37], and Roughtime [66]. The Roughtime client verifies the delegation, the signature, the validity window and the Merkle proof of every response, chains nonces across servers, and produces a malfeasance report when two signed responses are inconsistent; the intersection of the signed intervals gives an authenticated, if coarse, interval for the local clock error.

6 Software design

Figure 1 shows the structure. Parsers, estimators, event detectors, masks and instrument profiles are kept in registries that third-party packages extend through Python entry points. The public interface is one flat module whose surface (91 names and their signatures) is recorded in a snapshot that the test suite compares on every change; names are removed only after a deprecation period, so that scripts written for a study keep working across releases. Logs larger than 256 MB are read in blocks with bounded memory.

The only runtime dependency is NumPy [67]. This keeps installation simple, and it lets the same code run in a web browser through Pyodide [68], at the cost of implementing some numerical routines (the χ^2 quantiles, for instance) that a larger scientific stack would provide. The command-line interface has 33 commands. A local web interface written in plain JavaScript, with one bundled charting library [69] and no build step, is served by the Python standard library; the in-browser edition runs the same interface without a server. For operations, a monitor exports OpenMetrics and OpenTelemetry, and a GitHub Action and pytest helpers turn time-error, stability and audit checks into test assertions.

The package is MIT-licensed. Version 3.7.0 has about 16,400 lines of Python in 55 modules and 597 automated tests, run on Python 3.9, 3.11 and 3.13 together with static type checking, linting, the execution of the example notebooks and browser tests of the web interface. None of the tests needs network access; a separate weekly workflow queries public servers (Section 8.4).

7 Validation

7.1 Published reference values

NIST SP 1065 [7, sec. 12] publishes two test suites: the nine-point frequency data of NBS Monograph 140 (its Table 30, at averaging factors 1 and 2) and a 1000-point series generated by the linear congruential recursion $n_{i+1} = 16807\, n_i \bmod (2^{31} - 1)$ from $n_0 = 1234567890$ (Table 31, at factors 1, 10 and 100). Both are regenerated in the test suite and in the experiment script. The values printed in the handbook have seven significant digits. Table 3 lists the largest relative deviation of `ntpstats` from the printed values for every statistic in the tables; all are below the 5×10^{-7} that the rounding of seven digits allows, and the largest is 2.8×10^{-7} (TDEV, 1000-point suite). HTOT, which the handbook tabulates with a noise-dependent bias correction, agrees to within 0.3% in the test suite; we do not count it as an exact reproduction.

7.2 Coverage of confidence intervals

Agreement with tables checks the point estimates, not their uncertainty. To check the intervals, we simulated 400 records of 1025 phase points for each of the five power-law noise types, computed OADEV and MDEV with 95% intervals at $m = 1, 4, 16, 64$, and counted how often the interval contained the

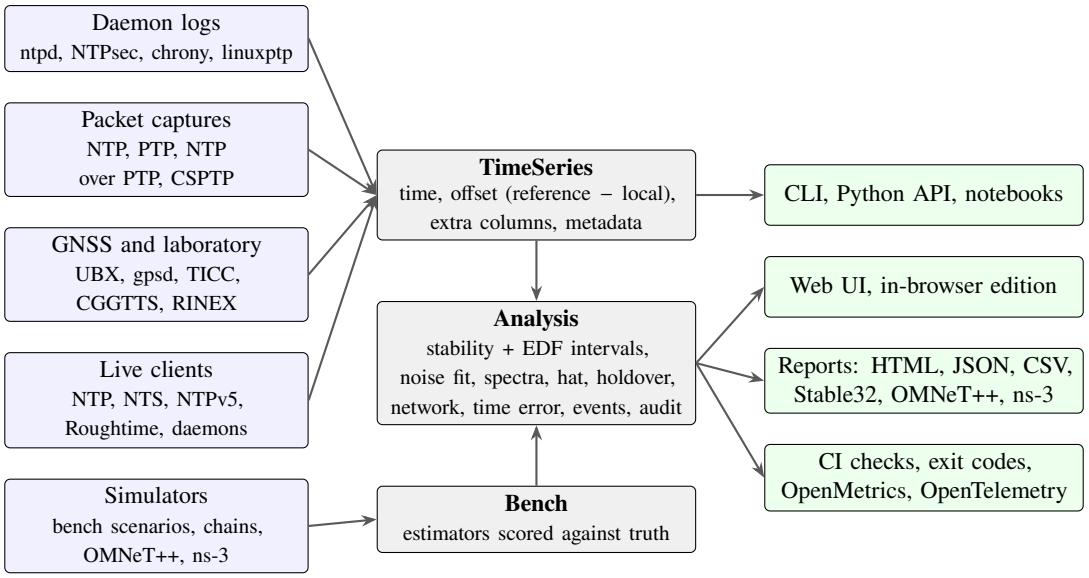


Figure 1: Structure of `ntpstats`. Every input is converted to one series type with one sign convention; the analysis and the bench operate only on that type, and every result is available through the command line, the Python API, the web interface and machine-readable reports.

Table 3: Largest relative deviation of `ntpstats` from the values printed in NIST SP 1065, Tables 30 and 31. The tables give seven significant digits, so deviations below 5×10^{-7} are rounding.

Statistic	NBS 9-point ($m = 1, 2$)	1000-point ($m = 1, 10, 100$)
ADEV	9.2×10^{-8}	8.5×10^{-8}
OADEV	2.8×10^{-9}	7.5×10^{-8}
MDEV	4.6×10^{-8}	7.5×10^{-8}
TDEV	5.0×10^{-8}	2.8×10^{-7}
HDEV (overlapping)	4.5×10^{-8}	9.9×10^{-8}
TOTDEV	5.6×10^{-9}	7.5×10^{-8}
MTOT	7.8×10^{-8}	9.6×10^{-8}
TTOT	7.0×10^{-8}	1.0×10^{-7}

expected deviation of the generating model. The noise type was identified from each record, as a user would do. Figure 2 shows the result next to the coverage of the common shortcut $\sigma \pm 1.96\,\sigma/\sqrt{n}$.

With 400 runs, the standard error of an observed coverage near 0.95 is about 0.011. With the exact EDF the coverage lies between 0.91 and 0.98 in 38 of the 40 cases, and between 0.93 and 0.97 in 34. The shortcut is adequate only at $m = 1$ and, for OADEV, under white PM; elsewhere its coverage falls to between 0.24 and 0.6 at $m = 64$, because overlapping terms are strongly correlated. The exception for the EDF method is flicker PM with OADEV at large m (0.87 at $m = 16$ and 0.70 at $m = 64$, against 0.93 and 0.92 for MDEV). For this noise type the Allan deviation depends only weakly on τ and the lag-1 estimate of the noise type becomes ambiguous on short records, so a wrong EDF model is chosen in part of the runs. MDEV, which separates white and flicker PM, does not show the problem. We report this as a limitation: users who expect flicker PM should prefer MDEV or TDEV, which is also the recommendation of the time-and-frequency literature [7].

Further tests, which we summarise without repeating their numbers, check every estimator against a literal implementation of its formula and against the analytic slopes of the five noise types; check the EDF against Monte Carlo variances; check the coverage of the noise-model intervals, the N-cornered hat and the holdover envelope; compare against a frozen table of values from an independent implementation (`allantools`, stored as data, not a dependency); and exercise the protocol clients against local test servers, including Kiss-o'-Death packets, spoofed and forged responses, certificate mismatches and the

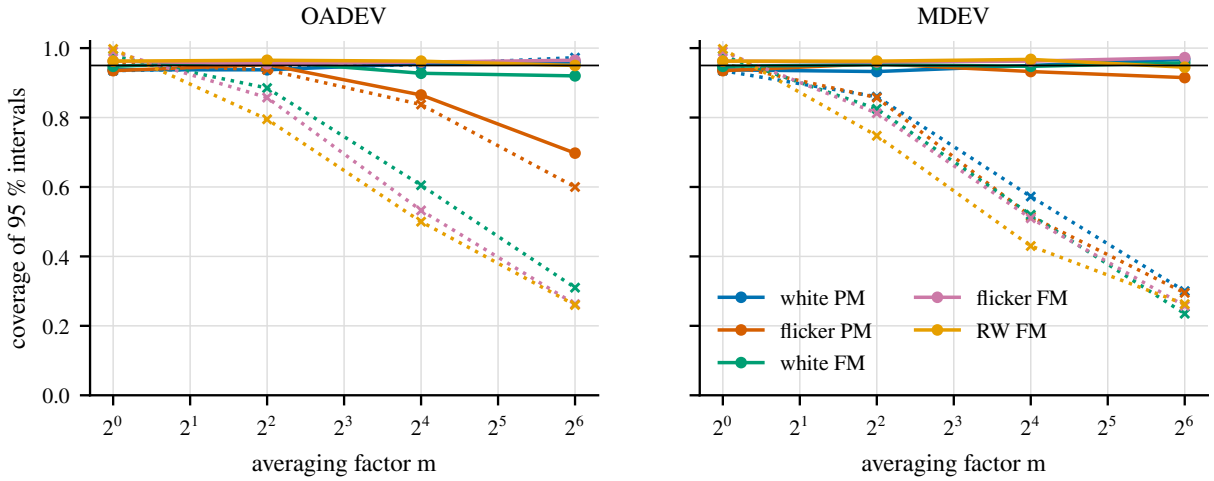


Figure 2: Share of 400 simulated records in which the 95% interval contains the true deviation. Solid: χ^2 intervals with the exact discrete EDF and the noise type identified from each record. Dotted: the shortcut $\sigma \pm 1.96\,\sigma/\sqrt{n}$. The horizontal line marks 0.95.

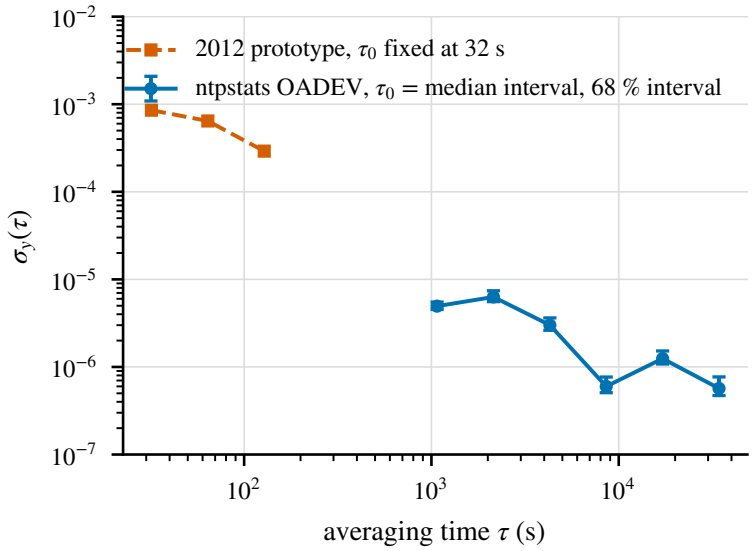


Figure 3: Stability of the 2012 sample `loopstats` file: the 2012 prototype, which fixed $\tau_0 = 32$ s, against `ntpstats` OADEV with the median interval, gaps left open and 68% intervals.

NTP era rollover of 2036.

7.3 A silent error in practice

The 2012 prototype of this project [20] is preserved unchanged in the repository, together with the ntpd `loopstats` file it was developed with. That file has 52 samples written at the poll interval, with a median spacing of 1072 s and a gap of 2.6 h. The prototype's Allan deviation routine assumed a spacing of 32 s regardless of the data and treated the samples as evenly spaced. Figure 3 compares its output with `ntpstats`. The prototype reports $\sigma_y = 8.6 \times 10^{-4}$ at $\tau = 32$ s. Rescaling to the actual spacing moves both axes by a factor of $1072/32 = 33.5$, which gives 2.6×10^{-5} at 1072 s; `ntpstats`, which in addition does not bridge the gap, gives 4.9×10^{-6} with a 68% interval of $[4.5, 5.5] \times 10^{-6}$. With 52 samples the curve says little about this clock, but the case shows how a plausible-looking stability plot can be wrong by two orders of magnitude when the sampling is assumed rather than read from the data.

Table 4: RMS error of the offset estimate (µs), mean ± standard deviation over five seeds, excluding the first 30 minutes. Single-server estimators, scored on the first server.

Estimator	lan	internet	congested	route-change
raw	26.1 ± 1.3	1610 ± 49	8900 ± 190	2160 ± 59
mindelay	1.08 ± 0.14	118 ± 21	1670 ± 50	1520 ± 29
kalman	5.56 ± 0.58	644 ± 32	3200 ± 250	1300 ± 60
kalman-dw	2.89 ± 0.14	177 ± 15	1100 ± 220	1430 ± 21
rts-dw	2.17 ± 0.085	211 ± 12	1240 ± 120	1300 ± 23
regression	3.99 ± 0.34	54.4 ± 13	575 ± 64	1330 ± 14
feedforward	8.10 ± 0.12	58.5 ± 5.9	578 ± 29	2330 ± 920
hull	2.50 ± 0.075	11.3 ± 0.11	12.7 ± 0.38	1380 ± 12
ptp4l	47.7 ± 3.2	3080 ± 110	13,600 ± 490	3020 ± 99
sptp	19.0 ± 2.9	1980 ± 93	12,500 ± 730	2430 ± 120
ntpd	16.1 ± 4.5	1120 ± 760	4960 ± 1400	2050 ± 790
ntpd-rfc	12,100 ± 4100	36,400 ± 28,000	49,100 ± 37,000	58,700 ± 19,000
lockclock	218 ± 4.1	711 ± 82	4600 ± 210	1630 ± 180
levine-kalman	218 ± 4.2	654 ± 89	5500 ± 1100	1460 ± 330

8 Experiments

All results in this section are produced by `experiments.py` and `make_tables.py`, published with this paper, from `ntpstats` 3.7.0 with fixed seeds.

8.1 Estimators on simulated NTP networks

Table 4 reports the RMS error of the single-server estimators on four of the bundled scenarios, over five seeds. `lan` is a local network (16 s polling, 80 µs propagation each way, tens of microseconds of queueing, 6 h); `internet` is a wide-area path (64 s polling, 5 ms propagation each way, 1 and 3 ms mean queueing, 24 h); `congested` has 20 ms propagation and 8 and 15 ms mean queueing with 2% loss; `route-change` adds a 4 ms step of the forward propagation delay after 8 h, an evening congestion period and a temperature cycle of the oscillator.

Three observations stand out. First, on paths whose two directions have the same floor, the choice of estimator matters by two orders of magnitude: on `internet` the raw offsets have an RMS error of 1.6 ms, the NTP clock filter 118 µs, regression and feed-forward estimators about 55 µs, and the Huygens-style hull 11 µs. The hull uses the bound $\theta \pm \delta/2$ of every exchange, not only the lowest-delay ones, which is why it gains most where queueing is heavy (13 µs on `congested`, against 575 µs for the next best). Second, PTP-style servos tuned for hardware time stamps on a local network (`ptp4l`, `sptp`) follow the delay variation of a wide-area path and are worse than the raw offsets there; they are included as references for the PTP experiments below, not as NTP clients. Third, no estimator survives the route change. After the step, every estimator settles at an error of about 2 ms, half of the 4 ms one-way step: the median error between 10 and 18 h, averaged over five seeds, is 2.00 ms for the clock filter, 1.99 ms for the hull and between 1.71 and 1.92 ms for the others. This is the asymmetry limit of Equation (1) made visible: the error is in the path, and no two-way estimator can see it.

The closed-loop clock disciplines sit between the raw offsets and the delay-aware estimators. The model of ntpd's loop reduces the raw error by 30 to 45% on `lan` (16.1 µs against 26.1 µs), `internet` (1.12 ms against 1.61 ms) and `congested` (4.96 ms against 8.90 ms), with a large spread between seeds that comes from its start without a drift file. During the start-up clamp ntpd does not sort its clock filter and takes each sample as it arrives [58], so the frequency it measures directly at the end of the first 300 s rests on one unfiltered offset; the error of that estimate is then removed only slowly by the loop. With the filter sorted from the start, as in an earlier version of the model, the reductions were 56 to 67%, and on `falseticker`, where the start-up error dominates, the model is now worse than the raw offsets (1.76 ms against 1.00 ms). Its popcorn spike suppressor never fired on these presets: the suppressor measures the

Table 5: Multi-server estimators on `falseticker` (five seeds): RMS error (µs) and largest absolute error (ms).

Estimator	RMS (µs)	max (ms)
`median`	14,300 ± 26	20.2
`rfc5905`	188 ± 56	1.23
`kalman-combine`	292 ± 14	0.394
`hull` (server a only)	11.3 ± 0.054	0.0200

Table 6: RMS error (ns) on exchange-level PTP, three seeds of 4 h.

Estimator	`ptp-lan`	`ptp-tc`
`raw`	2980	149
`ptp4l`	2590	129
`sptp`	2270	114
`kalman-dw`	344	22.5
`mindelay`	187	11.2
`hull`	6.99	6.98

jitter around the newly selected sample, so a spike that the filter selects raises the threshold it would have to exceed; `lockclock` and its Kalman variant reduce the raw error by 25 to 60% on the wide-area presets, but on `lan` their first 1000 s cycles dominate the RMS. The Kalman variant improves on `lockclock` on `internet` and `route-change` and not on `congested`; Levine reported a factor of two on noisy telephone links with σ_o calibrated separately. None of the loops is as accurate as the regression or hull estimators on the same exchanges. The variant with the constants of the RFC 5905 appendix is the clearest result: it is worse than the raw offsets on every preset, by factors from 5 (`congested`) to more than 400 (`lan`). The appendix slews the phase by $\theta/(\mathrm{PLL}\,\min(2^{\mathrm{poll}}, 1500\,\mathrm{s}))$ per second with PLL = 65,536, a time constant of about 48 days at poll 6, against 17 minutes with the gain of 16 used in ntpd; a 10 ms phase step of the reference is still 9.9 ms off after 10 hours, and after a 10 ppm frequency step the error grows to 85 ms in 10 hours. The constants contradict the RFC's own Figure 27, which gives a time-constant scale of 16 and an averaging constant of 8, as ntpd does; with those two values the appendix loop reproduces the step responses of the ntpd model to the printed digits. The appendix's popcorn spike suppressor has a units error of the same kind: it compares an interval in seconds with twice the poll exponent rather than twice the poll interval. Implementations follow ntpd, but the appendix code, taken literally, barely disciplines a clock; errata with the evidence are in the repository (`research/rfc5905-errata`).

The `falseticker` scenario has four servers: one with a constant 40 ms error and one that steps by 100 ms after 12 h, both larger than the root distance of about 10 ms. Table 5 shows that the RFC 5905 selection and the intersection-based combination reject them, whereas the median of the four clock-filter outputs follows them once two of the four servers are wrong. A single-server estimator on a correct server is, in this scenario, better still; the value of the selection algorithms lies in not knowing which server that is.

8.2 PTP exchanges and clients

The `ptp-lan` scenario models a PTP slave behind switches without PTP support: one Sync and one Delay_Req per second, 8 ns time-stamp noise, 4 µs propagation, and mean queueing of 3 µs towards the slave and 6 µs towards the master with loads of 0.4 and 0.5. `ptp-tc` is the same network with transparent clocks that correct 95% of the queueing. Figure 4 and Table 6 show the RMS error over three seeds of four hours.

With linuxptp's default gains at one Sync per second, the modelled `ptp4l` servo follows the packet delay variation: without on-path support it improves on the raw offsets by only 13%, and transparent clocks reduce its error twentyfold, from 2.6 µs to 129 ns. Estimators that use the delay of every exchange do far better on the same packets: the clock filter reaches 187 ns without on-path support, and the hull

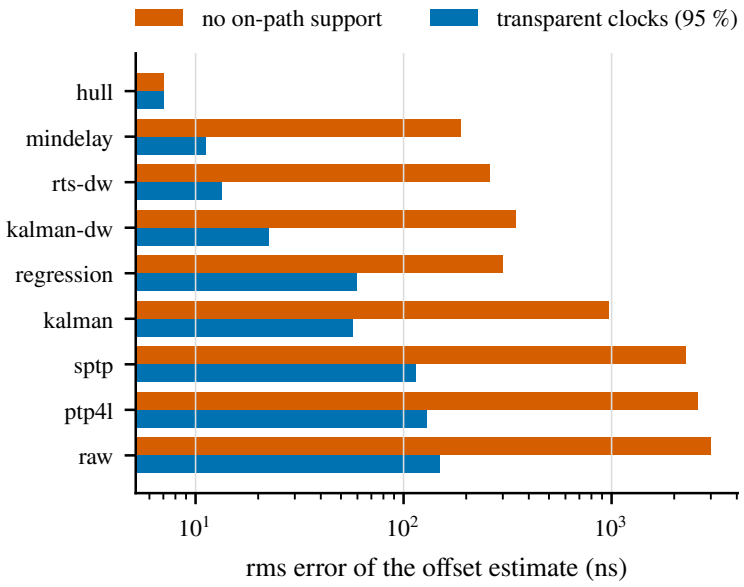


Figure 4: RMS error of the offset estimate on exchange-level PTP, without on-path support and with transparent clocks that correct 95% of the queueing (three seeds, 4 h each).

Table 7: Boundary-clock chains of 20 hops: max|TE| (ns) at hops 1, 5, 10 and 20, MTIE of dTE_L at hop 20 (ns), and the warm-up excluded (s). Mean of three seeds.

Servo	1	5	10	20	$MTIE_{20}$	warm-up
PI, linuxptp gains	6.0	16.0	56.9	728.2	732.3	300
PI, narrow loop	1.9	2.9	3.6	4.9	8.6	1250
linreg	29.0	70.3	182.1	580.7	166.5	300

estimator 7 ns in both cases, which is close to the time-stamp noise. The comparison has a caveat. The hull assumes that the two directions have equal floors, which holds in this scenario by construction; a constant asymmetry would bias it, like every two-way method. Within that assumption, the result supports the case for delay-filtering PTP clients and for carrying NTP over PTP, where corrections and hardware time stamps can be combined with a client-side estimator.

8.3 Time error along boundary-clock chains

We simulated chains of 20 boundary clocks at 16 Sync and 16 Delay_Req messages per second for one hour, with three servo configurations and three seeds each: the PI servo with linuxptp's default gains for hardware time stamping, the PI servo with a narrow loop ($k_p = 0.3$ s^{-1}, $k_i = 3.5 \times 10^{-4}$ per sample, a bandwidth of about 0.05 Hz), and the linear-regression servo. The warm-up excluded from the metrics is derived from the slowest pole of the loop, with a minimum of 300 s.

Figure 5 and Table 7 show the effect known as gain peaking. With the default gains, which are chosen for a single slave, each loop amplifies part of the wander it receives; cascaded, the peaks multiply, and max|TE| grows from 6 ns at the first hop to 57 ns at the tenth and 728 ns at the twentieth. A narrow, well-damped loop keeps the twentieth hop at 4.9 ns, at the price of a longer lock time (the excluded warm-up rises from 300 to 1250 s). The regression servo starts with a larger error (29 ns at the first hop) but grows more slowly, and at twenty hops it ends close to the default PI servo (581 ns). Under these assumptions (no asymmetry, no packet delay variation between boundary clocks, and oscillators with identical noise parameters), the default configuration uses two thirds of the 1.1 µs budget of G.8271.1 at twenty hops before any asymmetry is counted, whereas the narrow loop leaves almost all of it for asymmetry and other error sources. The model is not a product: real boundary clocks have their own

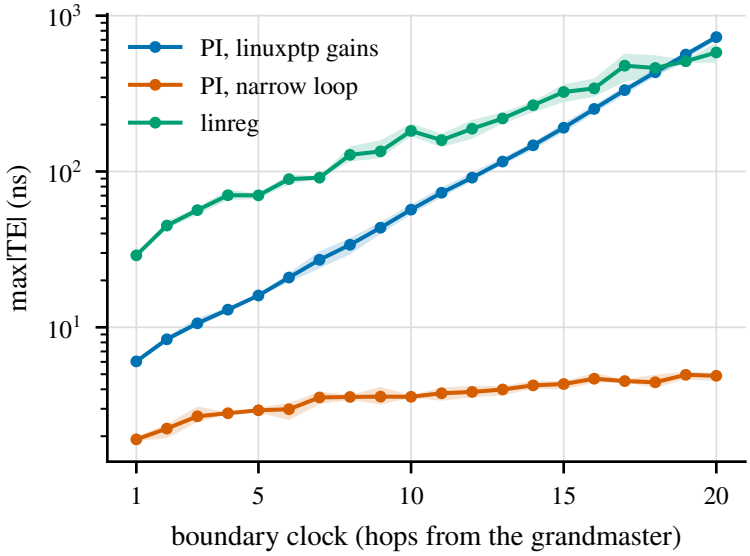


Figure 5: max|TE| of each boundary clock along a 20-hop chain (mean of three seeds; the band spans the seeds). Default linuxptp gains show gain peaking: the error grows much faster than the number of hops.

servos and filters. What the simulation provides is a controlled way to see how servo bandwidth, message rates, asymmetry and noise interact along a chain, and to check a design against a budget before building it.

8.4 Public servers

A weekly workflow on GitHub-hosted runners queries public servers with every client, and the first run of each month is added to an open dataset in the repository, archived with each release. The run of 30 September 2026 (ntpstats 2.13.0, IPv4, Table 8) illustrates what the data contain. All six NTPv4 servers and all five NTS servers answered; each NTS server returned a fresh cookie per exchange (eight cookies left after two exchanges); none of the six servers probed answered in interleaved mode, and none of four servers probed offered NTPv5. Three of the four Roughtime servers answered and one timed out; the NTS pool test failed because the experimental pool's name did not resolve. In every NTP and NTS measurement, $|\theta|$ was between 0.2% and 40% of $\delta/2$, so all offsets are compatible with a correct server and an asymmetric path. A single run is a snapshot, not a measurement of the servers' accuracy; the value of the dataset is in its growth and its openness. Because the runner's clock is not a reference and changes between runs, the trend view of the dataset takes each server's offset relative to the median of all NTP servers in the same run, which cancels the runner's error, and flags a server only when that relative offset moves by more than both runs' bounds $\delta/2$ plus 1 ms, a change that no path asymmetry can cause.

8.5 Revisiting the 2012 thesis

The thesis in which `ntpstats` began [20] set out to let NTP clients poll less often by modelling the local clock better. It analysed NTP's hybrid phase- and frequency-locked loop, discretised it, and proposed a correction loop in which the offsets measured against a NIST server were smoothed with a Savitzky–Golay filter [70] (fifth order, 39 samples) and then passed to a two-state Kalman filter, drawing on Levine's work on the NIST time services [71, 72]. The experiments polled `utcnist2.colorado.edu` every 32 s and were judged by the Allan deviation of the corrected offsets. The original code and two of the original measurement logs are kept in the repository, so the work can be checked; the script `research/thesis-2012/replicate.py` does so.

Three findings stand out. First, the smoothing class in the 2012 code defaults to returning the second derivative (`deriv=2`) and was instantiated without overriding it, so the stage that was meant to smooth the offsets returned their second derivative: on the December 2012 measurements it turns 16.4 ms RMS

Table 8: Single exchanges from one GitHub-hosted runner (30 September 2026). θ is the server minus the runner clock; the last column compares $|\theta|$ with the asymmetry bound $\delta/2$.

Test	Server	Stratum	θ (ms)	δ (ms)	$\lvert\theta\rvert/(\delta/2)$
NTPv4	`time.cloudflare.com`	3	+0.375	1.88	0.40
NTPv4	`time.google.com`	1	+0.298	9.31	0.06
NTPv4	`ptbtime1.ptb.de`	1	+0.235	90.65	0.01
NTPv4	`time.nist.gov`	1	-2.315	46.19	0.10
NTPv4	`pool.ntp.org`	2	+0.945	94.75	0.02
NTPv4	`ntp.ubuntu.com`	2	+0.984	76.48	0.03
NTS	`time.cloudflare.com`	3	+0.087	2.45	0.07
NTS	`nts.netnod.se`	1	+1.420	111.76	0.03
NTS	`ptbtime1.ptb.de`	1	+0.106	90.19	0.00
NTS	`ntppool1.time.nl`	1	+1.190	87.72	0.03
NTS	`ntp.3eck.net`	2	-1.721	97.95	0.04

Table 9: The 2012 correction loop against known truth (`internet` scenario, 32 s polling, 12 h, five seeds, first 30 minutes excluded): RMS error of the estimate and the Allan deviation of the estimate itself at 32 s, the quantity the thesis used. The true clock's is 3.0×10^{-10}.

Estimator	RMS error (µs)	OADEV of the estimate, 32 s
raw offsets	1610	8.0×10^{-5}
Savitzky–Golay 5/39, centred (offline)	791	3.2×10^{-6}
2012 Kalman filter	1300	4.3×10^{-5}
centred Savitzky–Golay + 2012 Kalman (design)	795	1.9×10^{-6}
as built (`deriv=2`) + 2012 Kalman	132,000	1.4×10^{-7}
`kalman-dw`	167	4.9×10^{-7}
`regression`	53.1	1.0×10^{-6}
`hull`	3.25	1.8×10^{-8}

into 0.13 ms, where the intended smoother leaves 11.8 ms. Second, the Allan deviation of an estimate does not measure its accuracy. On the same real data, a correct smoothing pass lowers $\sigma_y(32\,\mathrm{s})$ 25-fold and changes the noise type identified, without adding any information about the clock. Table 9 makes the point with known truth: on the `internet` scenario at 32 s polling, the pipeline as built produces an output whose Allan deviation at 32 s is 570 times lower than that of the raw offsets, the second lowest of all, while its error is about 80 times larger. The pipeline as designed halves the raw error, but its centred window needs 19 future samples, so it cannot run in real time; the estimators of Table 2 do far better on the same exchanges. Third, the thesis's first objective, a parameter linking the poll interval with the achievable accuracy, can now be answered. The May 2012 log is a free-running PC clock measured every 1024 s for five days; its frequency offset was 491.3 ppm and, three days later, 491.0 ppm. Its residual is network noise (8.5 ms RMS), and the noise fit detects no clock noise above it. Since version 3.6 the question is a command: for each candidate interval the log is decimated to that interval and the holdover model predicts the error just before the next poll, so that the fewer measurements of a longer interval count against it. For this clock and path the 95% bound is 21.7 ms at 17 minutes, 27.7 ms at 34 minutes and 37.9 ms at 68 minutes; the network dominates throughout. The thesis's control-theory chapters hold up: NTP needs a type II loop, and at NTP's ratio of loop time constant to update interval (about 32) forward Euler, backward Euler and Tustin discretisations place the closed-loop poles within 5×10^{-4} of the continuous system's, so the choice among them does not matter. Run against truth with the model of ntpd's discipline (Section 5), a 10 ms phase step is absorbed to below 1 ms within 35 minutes at poll 64 s, and a 10 ppm frequency step is removed completely, as a type II loop must, but only after about 10 hours.

8.6 Computational cost

On one core of the machine used for this paper (x86-64, Python 3.13, NumPy 2.5), OADEV, MDEV, HDEV and TOTDEV of a record of 10^6 phase points at octave-spaced averaging times, each with noise identification and EDF-based intervals, took between 0.9 and 1.6 s. The complete experiment script runs in about ten minutes.

9 Discussion

What the toolkit does not claim. The estimators and servos are models written from published descriptions, not the daemons' code; their numbers characterise algorithms under stated conditions, not products. The simulator's path model (a fixed floor with exponential queueing) is deliberately simple; trace replay is the way to bring real delay distributions into the bench, and it inherits the unobservability of a constant asymmetry. Parsers for vendor instruments rely on declarative profiles until sample exports are contributed. The validation reproduces published tables, checks interval coverage by simulation and compares with Stable32's output files, summarised below. A long comparison of real logs against an independent reference has not been done yet; the protocol for it is described at the end of this section.

Comparison with Stable32. Stable32 1.62 [16] runs under Wine without a display. Its batch command computes four statistics; its automation scripts, run the same way, compute the others. It was run on four phase files (the 1000-point test suite and the sample file it ships with, and two simulated power-law mixtures) for eleven statistics with its 68% intervals, 44 runs in all; the output files are in the repository with a test that compares them. For OADEV, MDEV, TDEV and HDEV the point estimates agree on all 160 rows to the four digits Stable32 prints and the numbers of analysis points are identical; $\mathrm{TIE}_{\mathrm{rms}}$ and MTIE agree to the printed digits as well. The intervals agree wherever the two programs use the same equivalent degrees of freedom. They differ in two places. At small averaging factors for FM noise Stable32's EDF follows Greenhall's algorithm for phase averaged over the sample interval and is larger than the exact value for instantaneous samples (782 against 666 for white FM at $m = 1$); the difference is gone by $m = 64$. When fewer than about 30 averaged points remain, Stable32 switches its noise identification to a B1-ratio method and on these records reports a different noise type than the generating model; its EDF is then `ntpstats`'s EDF for that other noise type to the printed digits, so the formula is the same and the identification differs.

The total and Theo statistics showed how much depends on conventions that output files do not state. Stable32 corrects TOTDEV and HTOT for the identified noise type, and wherever the two programs identify the same type the values agree to the printed digits; on the remaining rows each of its values equals `ntpstats`'s raw sum divided by the bias factor of another noise type, exactly. It writes MTOT and TTOT without the correction that its manual and SP 1065 describe, so these agree with `ntpstats`'s raw values. Its Theo1 run writes the bias-removed TheoBR, not Theo1; `ntpstats`'s TheoBR agrees with it to the printed digits on the 1000-point files, and on 4096 points the Theo1 sums agree at every τ while the bias-removal ratio differs by 0.1 to 0.2%. Three changes to `ntpstats` came out of the comparison: the χ^2 quantile, a Wilson–Hilferty approximation off by about half at two degrees of freedom for the 2.5% tail, is exact since version 3.6.1; TOTDEV is bias-corrected as SP 1065 prescribes since 3.7.0, which raises long-τ values by up to 11% for random-walk FM at $\tau = T/4$; and the TheoBR ratio, which averaged a subset of its terms, uses all of them on records of up to about 1550 points, which had moved the result by 0.2% on 1000 points.

Validation against an independent reference. Simulation and cross-checks test the statistics, not a deployment. The missing measurement is a long record of a real client against a reference it does not use. `ntpstats` 3.7.0 provides the protocol: a host disciplined over the network by chrony logs the pulse per second of a GNSS receiver as a reference clock that is never selected, which records the true error of the system clock to about a microsecond. The `validate` command then reports the clock's error, whether

the daemon's own maximum-error estimate held over every update interval, whether each server stayed within its correctness interval $\theta \pm \lambda$ of RFC 5905, and how every estimator of the bench performs on the real exchanges. The hardware costs less than 100 US dollars; the first campaign, of several weeks, is planned and will be reported with its data.

Threats to validity. The experiments use simulated networks whose parameters were chosen to be representative, not measured; results on real networks can differ, which is why the bench accepts replayed traces. The coverage study uses one record length (1025 points) and four averaging factors; the flicker PM shortfall shows that the identification step, not the EDF itself, can limit coverage on short records. The public-server table is a single run from one location, and the poll-interval result of Section 8.5 rests on one machine and one week of data. The clock-discipline models follow the published code closely enough to reproduce its behaviour, including the start-up phase in which ntpd uses each sample unfiltered, but they are not the daemon.

Why one toolkit. Most of the functions described here exist somewhere, in a stability program, a daemon, an instrument or a paper's evaluation code. Putting them on one data model has practical consequences that go beyond convenience. The sign convention is fixed once, so a chrony tracking log, a PTP capture and a counter export can be compared directly. The same statistics, with the same intervals, are applied to a deployment log and to an algorithm's output on simulated truth, so a result in the field and a result in a paper can be related. And because the bench, the scenarios and the seeds are open, a claim that an algorithm improves on another can be checked by anyone. The author's own 2012 thesis (Section 8.5) is a case in point: its central claim could not be checked when it was made, and is checkable now.

10 Availability and reproducibility

`ntpstats` is available under the MIT licence from PyPI and conda-forge (`pip install ntpstats`), with source, documentation and issue tracker at `https://github.com/thiagodefreitas/NetworkTime` and documentation at `https://thiagodefreitas.github.io/NetworkTime/`. Each release is archived on Zenodo [73]. The LaTeX source of this paper, the experiment script, the generated data and the figures are in the directory `research/preprint-2026` of the repository. To reproduce every number:

```
pip install ntpstats==3.7.0 matplotlib
python research/preprint-2026/experiments.py
python research/thesis-2012/replicate.py
python research/preprint-2026/make_tables.py
```

This preprint is distributed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0).

Use of AI tools

An AI coding assistant was used during the development of the software, the experiment scripts and the preparation of this manuscript, under the author's direction. The author designed the work, reviewed and tested all code, checked every number against the scripts that produce it, verified every reference, and takes full responsibility for the content.

Acknowledgements

`ntpstats` began as a Google Summer of Code project for the NTP Project of the Network Time Foundation and as the author's undergraduate thesis at the Universidade Federal de Campina Grande.

The author thanks Judah Levine (NIST), mentor of the Google Summer of Code projects, Harlan Stenn and Dave Hart of the NTP Project for their support at the time, and Antonio Marcus Nogueira Lima and Marcos Ricardo Alcântara Morais, advisor and examiner of the thesis. The author also thanks the maintainers of the protocols, daemons, standards and open data sets on which this work builds.

References

[1] D. L. Mills. Internet time synchronization: The Network Time Protocol. *IEEE Transactions on Communications*, 39(10):1482–1493, 1991. doi: 10.1109/26.103043.

[2] D. Mills, J. Martin, J. Burbank, and W. Kasch. Network time protocol version 4: Protocol and algorithms specification. RFC 5905, RFC Editor, June 2010. URL https://www.rfc-editor.org/info/rfc5905.

[3] D. Franke, D. Sibold, K. Teichel, M. Dansarie, and R. Sundblad. Network time security for the network time protocol. RFC 8915, RFC Editor, September 2020. URL https://www.rfc-editor.org/info/rfc8915.

[4] *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. IEEE, Piscataway, NJ, USA, 2020. IEEE Std 1588-2019 (approved 7 November 2019).

[5] D. W. Allan. Statistics of atomic frequency standards. *Proceedings of the IEEE*, 54(2):221–230, 1966. doi: 10.1109/PROC.1966.4634.

[6] James A. Barnes, Andrew R. Chi, Leonard S. Cutler, Daniel J. Healey, David B. Leeson, Thomas E. McGunigal, James A. Mullen, Warren L. Smith, Richard L. Sydnor, Robert F. C. Vessot, and Gernot M. R. Winkler. Characterization of frequency stability. *IEEE Transactions on Instrumentation and Measurement*, IM-20(2):105–120, May 1971. doi: 10.1109/TIM.1971.5570702.

[7] W. J. Riley. Handbook of frequency stability analysis. NIST Special Publication 1065, National Institute of Standards and Technology, Boulder, CO, July 2008. URL https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=50505.

[8] *IEEE Standard Definitions of Physical Quantities for Fundamental Frequency and Time Metrology: Random Instabilities*. IEEE, Piscataway, NJ, USA, 2022. IEEE Std 1139-2022.

[9] ITU-T. Definitions and terminology for synchronization networks. ITU-T Recommendation G.810, International Telecommunication Union, August 1996. URL https://www.itu.int/rec/T-REC-G.810.

[10] ITU-T. Definitions and terminology for synchronization in packet networks. ITU-T Recommendation G.8260, International Telecommunication Union, August 2026. URL https://www.itu.int/rec/T-REC-G.8260. Edition 08/2026 (prepublished).

[11] ITU-T. Network limits for time synchronization in packet networks with full timing support from the network. ITU-T Recommendation G.8271.1/Y.1366.1, International Telecommunication Union, November 2022. URL https://www.itu.int/rec/T-REC-G.8271.1.

[12] ITU-T. Timing characteristics of telecom boundary clocks and telecom time synchronous clocks for use with full timing support from the network. ITU-T Recommendation G.8273.2/Y.1368.2, International Telecommunication Union, June 2023. URL https://www.itu.int/rec/T-REC-G.8273.2.

[13] D. Veitch, J. Ridoux, and S. B. Korada. Robust synchronization of absolute and difference clocks over networks. *IEEE/ACM Transactions on Networking*, 17(2):417–430, April 2009. doi: 10.1109/TNET.2008.926505.

[14] Yilong Geng, Shiyu Liu, Zi Yin, Ashish Naik, Balaji Prabhakar, Mendel Rosenblum, and Amin Vahdat. Exploiting a natural network effect for scalable, fine-grained clock synchronization. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 81–94, Renton, WA, 2018. USENIX Association. ISBN 978-1-939133-01-4. URL https://www.usenix.org/conference/nsdi18/presentation/geng.

[15] Yuliang Li, Gautam Kumar, Hema Hariharan, Hassan Wassel, Peter Hochschild, Dave Platt, Simon Sabato, Minlan Yu, Nandita Dukkipati, Prashant Chandra, and Amin Vahdat. Sundial: Fault-tolerant clock synchronization for datacenters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1171–1186. USENIX Association, 2020. ISBN 978-1-939133-19-9. URL https://www.usenix.org/conference/osdi20/presentation/li-yuliang.

[16] William J. Riley. Stable32: Software for frequency stability analysis. IEEE UFFC Society, Frequency Control Software. URL https://ieee-uffc.org/software. Accessed 3 October 2026.

[17] John Miles. TimeLab. Software. URL http://www.ke5fx.com/timelab/readme.htm. Accessed 3 October 2026.

[18] Anders E. E. Wallin, Danny C. Price, Cantwell G. Carson, and Frédéric Meynadier. allantools: Allan deviation calculation. Astrophysics Source Code Library, record ascl:1804.021, 2018. URL https://github.com/aewallin/allantools. Accessed 3 October 2026.

[19] NTPsec Project. ntpviz: Make data visualizations from statistics files. NTPsec documentation. URL https://docs.ntpsec.org/latest/ntpviz.html. Accessed 3 October 2026.

[20] Thiago de Freitas Oliveira Araújo. Modelagem e análise de relógios locais para otimização de sincronismo horário em rede. Trabalho de Conclusão de Curso (B.Sc. Electrical Engineering), Universidade Federal de Campina Grande, Campina Grande, Brazil, December 2012. URL https://dspace.sti.ufcg.edu.br/handle/riufcg/18226. In Portuguese. Advisor: A. M. N. Lima.

[21] M. Lichvar. Network time protocol (NTP) over the precision time protocol (PTP). RFC 10030, RFC Editor, August 2026. URL https://www.rfc-editor.org/info/rfc10030.

[22] Miroslav Lichvar and Richard Curnow. chrony. Software. URL https://chrony-project.org/. Version 4.9 (released 27 August 2026).

[23] D. W. Allan and J. A. Barnes. A modified "Allan variance" with increased oscillator characterization ability. In *Proceedings of the Thirty Fifth Annual Frequency Control Symposium*, pages 470–475. IEEE, 1981. doi: 10.1109/FREQ.1981.200514.

[24] D. A. Howe. The total deviation approach to long-term characterization of frequency stability. *IEEE Transactions on Ultrasonics, Ferroelectrics and Frequency Control*, 47(5):1102–1110, September 2000. doi: 10.1109/58.869040.

[25] D. A. Howe and T. K. Peppler. Very long-term frequency stability: Estimation using a special-purpose statistic. In *Proceedings of the 2003 IEEE International Frequency Control Symposium and PDA Exhibition Jointly with the 17th European Frequency and Time Forum*, pages 233–238. IEEE, 2003. doi: 10.1109/FREQ.2003.1275095.

[26] D. A. Howe. ThêoH: A hybrid, high-confidence statistic that improves on the Allan deviation. *Metrologia*, 43(4):S322–S331, August 2006. doi: 10.1088/0026-1394/43/4/S20.

[27] C. A. Greenhall and W. J. Riley. Uncertainty of stability variances based on finite differences. In *Proceedings of the 35th Annual Precise Time and Time Interval (PTTI) Meeting*, pages 267–280, 2003. URL https://ntrs.nasa.gov/citations/20050061319.

[28] W. J. Riley and C. A. Greenhall. Power law noise identification using the lag 1 autocorrelation. In *18th European Frequency and Time Forum (EFTF 2004)*, pages 576–580. IEE, 2004. doi: 10.1049/cp:20040932.

[29] András Varga and Rudolf Hornig. An overview of the OMNeT++ simulation environment. In *Proceedings of the First International ICST Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*. ICST, 2008. doi: 10.4108/ICST.SIMUTOOLS2008.3027.

[30] INET framework. Software. URL https://inet.omnetpp.org/. Version 4.7.0 (released 7 July 2026).

[31] George F. Riley and Thomas R. Henderson. The ns-3 network simulator. In Klaus Wehrle, Mesut Güneş, and James Gross, editors, *Modeling and Tools for Network Simulation*, pages 15–34. Springer, Berlin, Heidelberg, 2010. ISBN 978-3-642-12330-6. doi: 10.1007/978-3-642-12331-3_2.

[32] The Linux PTP project (LinuxPTP). Software. URL https://linuxptp.nwtime.org/. Accessed 3 October 2026.

[33] Richard Cochran, Cristian Marinescu, and Christian Riesch. Synchronizing the Linux system time to a PTP hardware clock. In *2011 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, pages 87–92. IEEE, September 2011. doi: 10.1109/ISPCS.2011.6070158.

[34] ntpd-rs. GitHub repository pendulum-project/ntpd-rs. URL https://github.com/pendulum-project/ntpd-rs. Accessed 3 October 2026.

[35] time: Meta's time libraries such as NTP and PTP. GitHub repository facebook/time. URL https://github.com/facebook/time. Accessed 3 October 2026.

[36] gpsd: A GPS service daemon. Software. URL https://gpsd.io/. Accessed 3 October 2026.

[37] M. Lichvar and A. Malhotra. NTP interleaved modes. RFC 9769, RFC Editor, May 2025. URL https://www.rfc-editor.org/info/rfc9769.

[38] *u-blox ZED-F9T Interface Description*. u-blox, r02 edition, June 2019. URL https://content.u-blox.com/sites/default/files/ZED-F9T_InterfaceDescription_(UBX-18053584).pdf. Document number UBX-18053584, protocol version 29.00, 13 June 2019.

[39] *TAPR TICC Timestamping Counter Operation Manual*. Tucson Amateur Packet Radio Corporation, November 2016. URL https://web.tapr.org/~n8ur/TICC_Manual-11-23-16.pdf. Revised 23 November 2016.

[40] P. Defraigne and G. Petit. CGGTTS-version 2E: An extended standard for GNSS time transfer. *Metrologia*, 52(6):G1, 2015. doi: 10.1088/0026-1394/52/6/G1.

[41] RIPE NCC Staff. RIPE Atlas: A global internet measurement network. *The Internet Protocol Journal*, 18(3):2–26, September 2015. ISSN 1944-1134. URL https://ipj.dreamhosters.com/wp-content/uploads/2015/10/ipj18.3.pdf.

[42] ClockBound. GitHub repository aws/clock-bound. URL https://github.com/aws/clock-bound. Accessed 3 October 2026.

[43] N. J. Kasdin and T. Walter. Discrete simulation of power law noise (for oscillator stability evaluation). In *Proceedings of the 1992 IEEE Frequency Control Symposium*, pages 274–283. IEEE, 1992. doi: 10.1109/FREQ.1992.270003.

[44] N. J. Kasdin. Discrete simulation of colored noise and stochastic processes and $1/f^{\alpha}$ power law noise generation. *Proceedings of the IEEE*, 83(5):802–827, May 1995. doi: 10.1109/5.381848.

[45] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, March 1978. doi: 10.1214/aos/1176344136.

[46] P. Welch. The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, June 1967. doi: 10.1109/TAU.1967.1161901.

[47] D. J. Thomson. Spectrum estimation and harmonic analysis. *Proceedings of the IEEE*, 70(9): 1055–1096, 1982. doi: 10.1109/PROC.1982.12433.

[48] K. S. Riedel and A. Sidorenko. Minimum bias multiple taper spectral estimation. *IEEE Transactions on Signal Processing*, 43(1):188–195, 1995. doi: 10.1109/78.365298.

[49] J. E. Gray and D. W. Allan. A method for estimating the frequency stability of an individual oscillator. In *Proceedings of the 28th Annual Symposium on Frequency Control*, pages 243–246. IEEE, 1974. doi: 10.1109/FREQ.1974.200027.

[50] François Vernotte, Claudio Eligio Calosso, and Enrico Rubiola. Three-cornered hat versus Allan covariance. In *2016 IEEE International Frequency Control Symposium (IFCS)*, pages 1–6. IEEE, May 2016. doi: 10.1109/FCS.2016.7546784.

[51] European Commission. Commission delegated regulation (EU) 2017/574 of 7 june 2016 supplementing directive 2014/65/EU of the European Parliament and of the Council with regard to regulatory technical standards for the level of accuracy of business clocks. Official Journal of the European Union, L 87, 31.3.2017, pp. 148–151, 2017. URL `http://data.europa.eu/eli/reg_del/2017/574/oj`.

[52] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.

[53] H. Theil. A rank-invariant method of linear and polynomial regression analysis, I–II: Confidence regions for the parameters of linear regression equations in two, three and more variables. *Indagationes Mathematicae*, 1950. URL `https://ir.cwi.nl/pub/18446`.

[54] Pranab Kumar Sen. Estimates of the regression coefficient based on Kendall's tau. *Journal of the American Statistical Association*, 63(324):1379–1389, December 1968. doi: 10.1080/01621459.1968.10480934.

[55] Hans R. Kunsch. The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3):1217–1241, September 1989. doi: 10.1214/aos/1176347265.

[56] Oleg Obleukhov and Ahmad Byagowi. Simple precision time protocol at Meta. Engineering at Meta (blog), February 2024. URL `https://engineering.fb.com/2024/02/07/production-engineering/simple-precision-time-protocol-sptp-meta/`. Accessed 3 October 2026.

[57] Network Time Foundation. ntpd/ntp_loopfilter.c, NTP reference implementation 4.2.8. `https://github.com/ntp-project/ntp`, . Accessed 3 October 2026.

[58] Network Time Foundation. ntpd/ntp_proto.c, NTP reference implementation 4.2.8. `https://github.com/ntp-project/ntp`, . Accessed 8 October 2026.

[59] Judah Levine. The statistics of computer clocks and the design of synchronization algorithms. *Journal of Research of the National Institute of Standards and Technology*, 125:125008, 2020. doi: 10.6028/jres.125.008.

[60] Judah Levine. Synchronizing computer clocks by the use of Kalman filters. In *Proceedings of the 43rd Annual Precise Time and Time Interval Systems and Applications Meeting (PTTI)*, pages 71–80, 2011.

[61] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, March 1960. doi: 10.1115/1.3662552.

[62] H. E. Rauch, F. Tung, and C. T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, August 1965. doi: 10.2514/3.3166.

[63] Keith Marzullo and Susan Owicki. Maintaining the time in a distributed system. *ACM SIGOPS Operating Systems Review*, 19(3):44–54, July 1985. doi: 10.1145/850776.850780.

[64] David Venhoek, Ruben Nijveld, and Marc Schoolderman. NTS extensions for enabling pools. Internet-Draft draft-ietf-ntp-nts-keyexchange-pool-01, July 2026. URL `https://datatracker.ietf.org/doc/draft-ietf-ntp-nts-keyexchange-pool/01/`. Work in Progress.

[65] Miroslav Lichvar and Tal Mizrahi. Network time protocol version 5. Internet-Draft draft-ietf-ntp-ntpv5-09, July 2026. URL `https://datatracker.ietf.org/doc/draft-ietf-ntp-ntpv5/09/`. Work in Progress.

[66] Watson Ladd and Marcus Dansarie. Roughtime. Internet-Draft draft-ietf-ntp-roughtime-19, March 2026. URL `https://datatracker.ietf.org/doc/draft-ietf-ntp-roughtime/19/`. Work in Progress.

[67] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2.

[68] Hood Chatham, Michael Droettboom, Gyeongjae Choi, Roman Yurchak, Dexter Chua, et al. Pyodide. Zenodo. URL `https://pyodide.org/`. Accessed 3 October 2026.

[69] Leon Sorokin. uPlot. GitHub repository leeoniya/uPlot. URL `https://github.com/leeoniya/uPlot`. Accessed 3 October 2026.

[70] Abraham Savitzky and Marcel J. E. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8):1627–1639, 1964. doi: 10.1021/ac60214a047.

[71] Judah Levine. An algorithm to synchronize the time of a computer to universal time. *IEEE/ACM Transactions on Networking*, 3(1):42–50, 1995. doi: 10.1109/90.365436.

[72] Judah Levine. Time synchronization over the Internet using an adaptive frequency-locked loop. *IEEE Transactions on Ultrasonics, Ferroelectrics and Frequency Control*, 46(4):888–896, July 1999. doi: 10.1109/58.775655.

[73] Thiago de Freitas. ntpstats, 2026. URL `https://github.com/thiagodefreitas/NetworkTime`.